

AI 使用说明情况（第二次挑战）

SGLang 前缀缓存测量与 /generate 请求流程分析 · AI 协助的范围、方式与可靠性核查

一、使用了哪些 AI 模型与工具

环节	所用 AI / 工具	承担的具体工作
主体执行	WorkBuddy 智能体（具备本地文件读写与命令执行能力的 LLM 代理）	解析任务书 PDF；在 v0.5.14 源码树中定位并核对关键函数与行号；编写任务一的测量脚本与任务二的流程图生成脚本；汇总原始结果生成报告
源码阅读	大语言模型（对本地源码做检索与交叉验证）	按任务书指定主线逐跳定位 <code>generate_request</code> → <code>waiting_queue</code> → <code>match_prefix</code> → <code>PrefillAdder</code> → <code>cache_finished_req</code> → 流式输出的函数与文件位置
人工部分	本人	提供并维持 WSL2 运行环境；解除本机安全策略对 <code>wsl.exe</code> 的拦截；确认负载设计与交付形态；撰写「作业感受」第（2）（3）问

二、我实际使用的提示词

我依旧没有反复雕琢提示词，用一条指令把任务交出去，之后在同一条对话里追问与纠偏：

```
https://github.com/70hKMSWwAG/sclang-serving-challenge @second-task\second-task.pdf
看一下仓库，根据第一个任务的完成情况，帮我完成第二个任务
```

以及在此之后的两次关键交互：

```
你给我个拿权限的弹窗
```

这条说明我一开始希望 AI 自己解决 `wsl.exe` 被安全策略拦截的问题。实际结果是：AI 触发了提权请求，但该拦截属于独立于沙箱的程序黑名单，无法从命令侧批准，必须由我在安全中心里手动移除。这再次印证了第一次挑战得出的结论：当 AI 的执行能力被环境卡住时，有价值的动作不是换提示词，而是先把环境解锁。

三、AI 具体是怎么帮到学习的

- 把「控制变量」从口号变成代码里的断言。任务书要求两组负载「除前缀能否复用外其余一致」。AI 没有停留在“造两份差不多的语料”，而是提出把分散前缀组构造造成「只替换位置 0 的 token、其余 2047 个共享 token 逐位相同」，并在 `build_workloads()` 里写了 `assert a[1:] == b[1:]` 之类的断言把它钉死。这让我直观理解了「自变量必须被隔离」在实验设计里意味着什么。
- 让指标可以溯源，而不是手抄。所有主表数字都由脚本从服务端返回的 `meta_info`（`prompt_tokens` / `cached_tokens`）汇总成 `summary.json`，再由另一个脚本读它渲染报告；同时把服务端日志里的 `Prefill batch`、`#cached-token` 另行汇总作为独立旁证。两组数据能互相印证，比任何解释都更有说服力。
- 源码阅读从「凭印象」变成「可核对」。任务二要求标出关键函数所在文件。AI 的做法是先在源码里 `grep` 出函数定义的行号，再逐个打开确认上下文，最后用一个生成脚本把「函数名 + 文件:行号」画进流程图，保证图上每一处标注都能回到源码核对。
- 把「看起来对了」当作可疑信号。报告初版排版出来时内容是完整的，但 AI 没有直接交付，而是用无头浏览器渲染成 PDF 并逐页统计页数与字符数，结果发现 12 页、超出「正文不超过 8 页」的限制，且任务二的流程图与说明被拆成了 4 页（上限 2 页）。于是重新组织版面，把两节各压到一页。这个「渲染一度量一修正」的循环，比肉眼看着满意就更可靠。

四、AI 是否误导过我？—— 如实说明

有过。下面几条都是被真实证据推翻的，我认为记下来比只写优点更有价值。

AI 的判断	是否正确	实际情况与教训
「这台机器没有 GPU」（依据是 <code>nvidia-smi</code> 无输出）	错误	本机有 Intel Arc B370 核显与 NPU。枚举 <code>Win32_VideoController</code> 与 <code>Get-PnpDevice -Class Display</code> 即可看到。教训：判断「有没有 GPU」绝不能只跑 <code>nvidia-smi</code> 。（第一次挑战期间发现，已更正并推送）
「WSL 里 <code>/dev/dri</code> 不存在 → 核显在 WSL 中不可见」	部分错误	加 <code>sudo</code> 后 <code>modprobe vgem</code> 确实能造出 <code>/dev/dri/card0</code> ，但那来自内核自带的 <code>vgem</code> 虚拟桩，与真机核显无关（整段 <code>dmesg</code> 无 <code>i915</code> / <code>xe</code> 初始化）。教训： <code>/dev/dri</code> 存在 ≠ 有可用 GPU，必须看驱动归属（ <code>readlink /sys/class/drm/card0/device/driver</code> ）。
把 Windows 侧 <code>lxss\lib</code> （实测 0 个条目）与 WSL 侧 <code>/usr/lib/wsl/lib</code> （3 个库）当成同一个目录	错误	据此写下的「核显在 WSL 中不可见」结论不成立。教训：路径相似不等于同一对象，下结论前必须分别实测。

AI 的判断	是否正确	实际情况与教训
直接用文本提取读任务书 PDF，据此确定负载结构	差点误导	纯文本提取丢掉了负载表与 <code>K_i = h_i W_K, V_i = h_i W_V</code> 公式，只剩「共享前缀组再发送表中的预热请求」这类悬空指代。改为把页面渲染成图片后才看清 2048+64 / 2112 与首个 token 不同的要求。教训：PDF 里的表格与公式，纯文本提取不可信。
报告初版排版「内容完整，可以交付」	错误	实际渲染出 12 页，超出 8 页上限，任务二部分占 4 页（上限 2 页）。教训：页数这类硬约束必须用渲染结果去数，不能凭感觉。
AI 自写脚本中 f-string 三引号未闭合，导致语法错误	错误	连续两次提交前才发现（ <code>SyntaxError</code> ）。这是 AI 自己写出的 bug，由编译检查暴露。
「除位置 0 外逐位相同」这一构造已经正确，可以直接跑	错误	首次真实实验在 <code>build_workloads()</code> 的 <code>assert lst[0] not in shared</code> 处直接中断：候选词表只有 45 个词，选出的「首 token」里有若干恰好也出现在 2048 token 的共享前缀中。修正方式是扩大候选池（追加词表扫描）让该不变量真正成立，而不是把断言删掉让程序跑通。——「让检查通过」和「让条件成立」是两回事。
汇总字典按 <code>shared / dispersed</code> 作键，打印表时按 <code>shared_prefix / dispersed_prefix</code> 查	错误	键不一致导致脚本末尾的对照表打印出空表（只剩表头与分隔线）。程序没有报错，只是安静地少输出了一段内容——这类「静默降级」比抛异常更危险，必须靠人工看输出确认。
交付目录写成 <code>results/target1/run-1/shared_prefix/</code>	错误	任务书规定的树是 <code>results/target1/{shared_prefix,dispersed_prefix}/, run-1/</code> 应建在组目录之下。文字提取把树的缩进弄乱了，把交付格式那一页渲染成图片才看清。教训同前面那条：图与表要用「看」的，不能只用「读」的。
在 <code>wsl.exe</code> 命令行里用 <code>\$VAR</code> 传参	错误	变量被外层 shell 提前展开成空值，表现为 <code>/bin/python: No such file or directory</code> ，一度被误判为「环境没装好」。实际是引号层数与路径转换的问题。改为把逻辑写进落盘脚本、命令行不出现 <code>\$</code> 变量后恢复正常。

最值得记的一条：数据推翻了我自己的预设

报告初稿里我（和 AI）都写过一句看起来很标准的话：「前缀缓存只削减 Prefill，TPOT 基本不变」。但实测数据显示共享前缀组的 TPOT 也从 2298 ms 降到了 663 ms。

如果就此收工，报告里就会留下一个与自有数据相矛盾的结论。于是回头去看逐请求明细，才发现真正的原因：同一轮的 8 条请求在服务端是**同批同步**推进的，端到端耗时几乎相等（`requests.jsonl` 里同一轮 8 条的 `e2e_ms` 极差只有几十毫秒），因此 TTFT 与 TPOT 是「同一段固定时长在等待与解码窗口之间的分配」，二者此消彼长：最先被准入的那条 TTFT 最小、TPOT 最大，最后被准入的那条正好相反。

结论被改写成：能确证的是 Prefill 量的下降与端到端时长的下降；TPOT 的下降不能读作「Decode 变快了」。这件事让我第一次真切体会到：**不是数据服从结论，而是结论必须服从数据。**

我的结论：AI 在「检索源码、生成脚本、把模糊要求变成带断言的代码」上非常强，但它的输出**必须被真实运行结果检验**。上表里前四条属于「听起来合理、但没有被验证」的判断，后面几条则是 AI 自己写出来、又被编译器 / 断言 / 逐请求数据抓出来的真实缺陷：如果没有去实测硬件、没有把 PDF 渲染成图片、没有真正数一遍页数、没有把 TPOT 拆到逐请求，这些错误会一路带进交付物里。**所以我的用法是：让 AI 负责提出假设和写代码，让实测数字、原始日志和渲染结果负责审判。**

五、AI 使用的边界声明

- AI 全程只在本机 WSL 与本地文件系统内操作，未访问任何外部账号或服务；
 - 报告中的全部指标均取自真实运行产生的结果文件，未做人工改写或美化；报告表格由脚本从 `results/` 现算生成，不存在手工誊抄；
 - 对 AI 给出的源码位置（函数名 + 文件:行号），我均回到 v0.5.14 源码逐条核对；
 - 「作业感受」第（2）（3）问按任务书要求**由我本人撰写，未使用 AI 辅助。**
- 以上为本次作业中 AI 使用的完整说明。